Tipi di dato derivati: gli Array

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



Indice dei Contenuti

1 Array Monodimensionali

2 Array Bidimensionali

3 Array Tridimensionali e n-dimensionali

4 Utilizzo degli Array

5 Array e Stringhe

Roadmap

1 Array Monodimensionali

2 Array Bidimensionali

3 Array Tridimensionali e n-dimensionali

4 Utilizzo degli Array

5 Array e Stringhe

Tipi di Dati Derivati e Aggregati

È possibile definire tipi di dati derivati costruiti a partire dai tipi nativi (int, $char$, $float$, ecc.).
Una prima categoria fondamentale è quella dei dati *aggregati*: collezioni di oggetti correlati.

Tipi di Dati Derivati e Aggregati

È possibile definire tipi di dati derivati costruiti a partire dai tipi nativi (int, $char$, $float$, ecc.).
Una prima categoria fondamentale è quella dei dati *aggregati*: collezioni di oggetti correlati.

Definizione di Array

Un **array** (o vettore) è una struttura dati omogenea e di lunghezza statica costituita da una sequenza contigua di elementi che:

- Hanno tutti lo **stesso tipo base** (es. tutti int, tutti $float$);
- Occupano posizioni di **memoria adiacenti**;
- Sono identificati univocamente da uno o più **indici numerici**.

Il numero di indici necessari definisce la *dimensione* dell'array (1D, 2D, n-D).

Array Monodimensionali (1D)

```c
#include <stdio.h>
#define N 100
int main(int argc, char *argv[])
{
  int a[N];

  for (int i=0; i<N; i++)
     a[i]=i;

}
```

Proprietà fondamentali:

- **Indicizzazione zero-based**: Gli indici vanno da 0 a $N-1$.
- **Accesso Diretto** $O(1)$: Si accede a qualsiasi elemento istantaneamente sapendo l'indice.
- **Dimensione Fissa**: Determinata al momento della compilazione (salvo VLA in C99).

Sintassi di Inizializzazione (C11 / Standard C)

Esistono diversi modi per inizializzare un array monodimensionale:

- **Completa esplicita**:

```
int arr[5] = {10, 20, 30, 40, 50};
```

- **Dimensione dedotta automaticamente**:

```
int arr[] = {1, 2, 3, 4}; // La dimensione allocata sarà 4
```

- **Inizializzazione parziale** (gli elementi rimanenti sono impostati a 0):

```
int arr[5] = {10, 20}; // {10, 20, 0, 0, 0}
int arr[5] = {0};      // Azzera tutti gli elementi
```

- **Designated Initializers (C99)**:

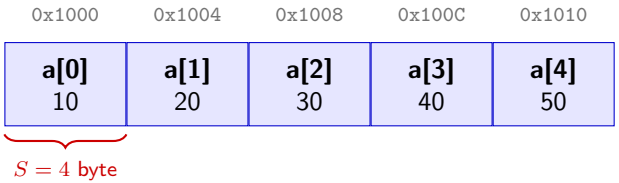
```
int arr[10] = {[2] = 5, [8] = 12}; // Inizializza solo gli indici 2 e 8
```

Mappatura in Memoria di un Array 1D

Essendo la memoria lineare, l'indirizzo di memoria dell'elemento i-esimo si calcola mediante una formula matematica diretta:

$$\text{Indirizzo}(a[i]) = \text{Indirizzo Base}(a) + (i \times S)$$

dove $S = \text{sizeof(tipo base)}$ espresso in byte.



Roadmap

1. Array Monodimensionali
2. Array Bidimensionali
3. Array Tridimensionali e n-dimensionali
4. Utilizzo degli Array
5. Array e Stringhe

Array Bidimensionali (2D)

Un array bidimensionale può essere concettualmente visto come una matrice organizzata in righe e colonne, o come un "array di array".

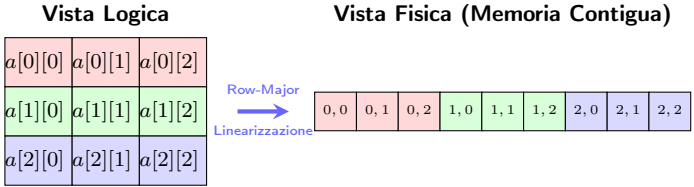
```
1  #include <stdio.h>
2  #define M 50
3  #define N 100
4  int main(int argc, char *argv[])
5  {
6    int a[M][N];
7
8    for (int i=0; i<M; i++)
9      for (int j=0; j<N; j++)
10       a[i][j]=N*i+j;
11
12 }
```

- Il primo elemento corrisponde all'indice $[0][0]$ (e l'ultimo a $[M-1][N-1]$);
- La memorizzazione avviene per *righe*;

Memorizzazione Per Righe (Row Major)

In C, la memoria fisica è unidimensionale. Gli array multidimensionali vengono salvati in ordine **Row-Major** (le righe vengono memorizzate consecutivamente).

$$\text{Indirizzo}(a[i][j]) = \text{Indirizzo Base}(a) + ((i \times N) + j) \times S$$



Roadmap

1 Array Monodimensionali

2 Array Bidimensionali

3 Array Tridimensionali e n-dimensionali

4 Utilizzo degli Array

5 Array e Stringhe

Array Tridimensionali e Generici n-D

È possibile estendere il concetto a 3 o più dimensioni (es. $N_1 \times N_2 \times N_3$).

```
1 // Array 3D: ad esempio 2 blocchi, ciascuno da 3 righe e 4 colonne
2 int tensor[2][3][4];
```

Generalizzazione della formula degli indirizzi per array 3D $A[D_1][D_2][D_3]$:

$$\text{Indirizzo}(A[i][j][k]) = \text{Base} + (i \times (D_2 \times D_3) + j \times D_3 + k) \times S$$

Riguardo le prestazioni

Poiché il C memorizza i dati in Row-Major, nei cicli annidati l'ordine in cui si scorre l'array con gli indici è importante!

Roadmap

1. Array Monodimensionali
2. Array Bidimensionali
3. Array Tridimensionali e n-dimensionali
4. Utilizzo degli Array
5. Array e Stringhe

Passaggio di Array a Funzioni

In C gli array non vengono **mai** passati per valore: il nome dell'array "decade" sempre all'indirizzo del suo primo elemento.
Per le funzioni 1D, le due sintassi seguenti sono **perfettamente equivalenti**:

```c
void print1D(int v[], int n);  // Sintassi con quadre (equivalente)
void print1D(int *v, int n);  // Sintassi con puntatore (equivalente)
```

1. Array 2D (Approccio Classico/Puntatori)

```c
// Senza VLA occorre passare un puntatore
// o specificare le colonne fisse: a[][N]
void print2D(int *arr, int m, int n) {
    for(int i=0; i<m; i++)
        for(int j=0; j<n; j++)
            printf("%d ", arr[i*n + j]);
}
```

2. Array 2D (Sintassi VLA da C99 in poi)

```c
// Le dimensioni devono precedere l'array!
void fill2D(int m, int n, int a[m][n]) {
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++)
            a[i][j] = i + j;
}

int main(void) {
    int A[3][4];
    fill2D(3, 4, A);
}
```

Vantaggi e Svantaggi degli Array

Vantaggi

- **Accesso Casuale** $O(1)$: Accesso diretto a qualsiasi elemento tramite indice e calcolo dell'indirizzo base.
- **Efficienza di Memoria**: Zero overhead di gestione (nessuna variabile extra da salvare)[a].
- **Località Spaziale**: Gli elementi contigui in RAM sfruttano al massimo la memoria Cache della CPU[b].

[a] lo vedremo nelle prossime lezioni!

[b] concetto che vedrete in Calcolatori Elettronici e/o Sistemi Operativi

Svantaggi / Limiti

- **Dimensione Fissa**: Allocazione rigida alla dichiarazione; impossibile ridimensionarli senza ri-allocazione.
- **Gli Elementi sono Omogenei**: Solo un tipo alla volta è utilizzabile dentro un array [a].
- **Costo Operazioni** $O(N)$: Inserimento e rimozione in mezzo/testa richiedono lo shift di N elementi.

[a] lo vedremo nelle prossime lezioni!

Roadmap

1 Array Monodimensionali

2 Array Bidimensionali

3 Array Tridimensionali e n-dimensionali

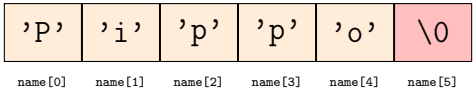
4 Utilizzo degli Array

5 Array e Stringhe

Array e Stringhe

Nel linguaggio C non esiste un tipo di dato nativo "stringa". Una stringa è rappresentata semplicemente come un array di caratteri terminato dal carattere nullo “” (ASCII 0).

```c
char name[6] = {'P', 'i', 'p', 'p', 'o', '\0'};
// Oppure in forma abbreviata (il compilatore aggiunge '\0' in automatico):
char name[] = "Pippo";
```



Input/Output di Stringhe in C

```c
#include <stdio.h>

int main() {
    char str[50];

    printf("Inserisci una parola: ");
    scanf("%49s", str); // Si ferma al primo spazio bianco!

    // Per leggere un'intera riga inclusi gli spazi:
    printf("Inserisci una frase: ");
    getchar(); // Consuma il '\n' residuo
    fgets(str, sizeof(str), stdin); // Sicuro: previene buffer overflow!

    printf("Hai scritto: %s", str);
    return 0;
}
```

Regola di Sicurezza

Non usare mai la funzione deprecata ‘gets()‘. Utilizzare sempre ‘fgets()‘ specificando la dimensione massima del buffer.

Operazioni sulle Stringhe (‘<string.h>‘)

Il linguaggio C offre standard library functions per manipolare le stringhe:

Funzione	Sintassi	Descrizione
`strlen`	`strlen(s)`	Restituisce la lunghezza della stringa (escluso `'\0'`).
`strcpy`	`strcpy(dest, src)`	Copia `src` in `dest` (incluso `'\0'`).
`strncpy`	`strncpy(dest, src, n)`	Copia fino a `n` caratteri (sicura).
`strcat`	`strcat(dest, src)`	Concatena `src` in coda a `dest`.
`strcmp`	`strcmp(s1, s2)`	Confronta due stringhe (restituisce 0 se identiche, < 0 o > 0 in ordine alfabetico).

```c
if (strcmp(str1, str2) == 0) {
    printf("Le stringhe sono uguali!\n");
}
```